The System Engineer Simulation Guide

How to use Python to run a simulation bench and write integration tests

asynchronics

Today's plan

- Recap
- Python API overview
- Real setup example
- Best practices

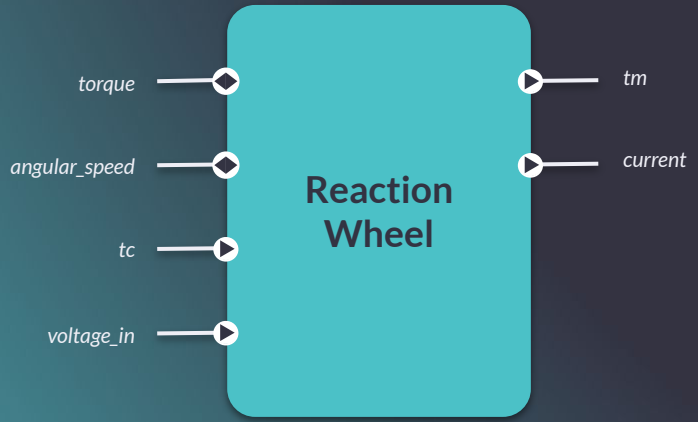
asynchronics

RECAP

asynchronics

Model interfaces are defined in terms of **ports**:

- **Input ports:** methods that implement the `InputFn` trait (return nothing).
- **Replier ports:** methods that implement the `ReplierFn` trait (return a reply).
- **Output ports:** member variables of the `Output` type.
- **Requestor ports:** member variables of the `Requestor` or `UniRequestor` type.

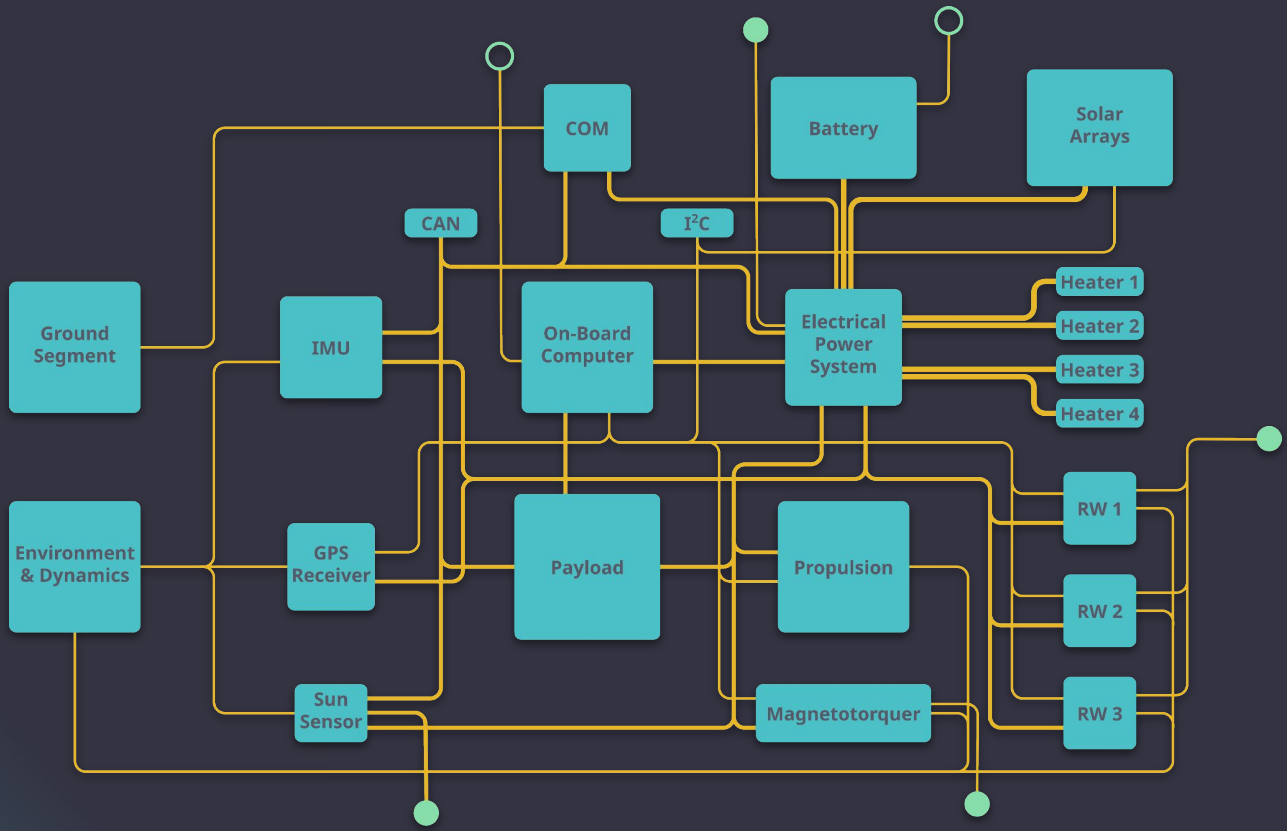



```rust
#[derive(Serialize, Deserialize, Default)]
struct ReactionWheel {
    tm: Output<TmPacket>,
    current: Output<f32>,
}

#[Model]
impl ReactionWheel {
    fn torque(&mut self) -> f32 {
        // ...
    }

    fn angular_speed(&mut self) -> f32 {
        // ...
    }

    fn tc(&mut self, packet: TcPacket) {
        // ...
    }

    fn voltage_in(&mut self, value: f32) {
        // ...
    }
}
```

asynchronics

Models are connected and the simulation exposes **endpoints**:

- **Event sources:** connected to one or several model inputs.
- **Query sources:** connected to one or several model repliers.
- **Event slots:** connected to one or several model outputs; store only the last event.
- **Event queues:** same as slots but can store many events.



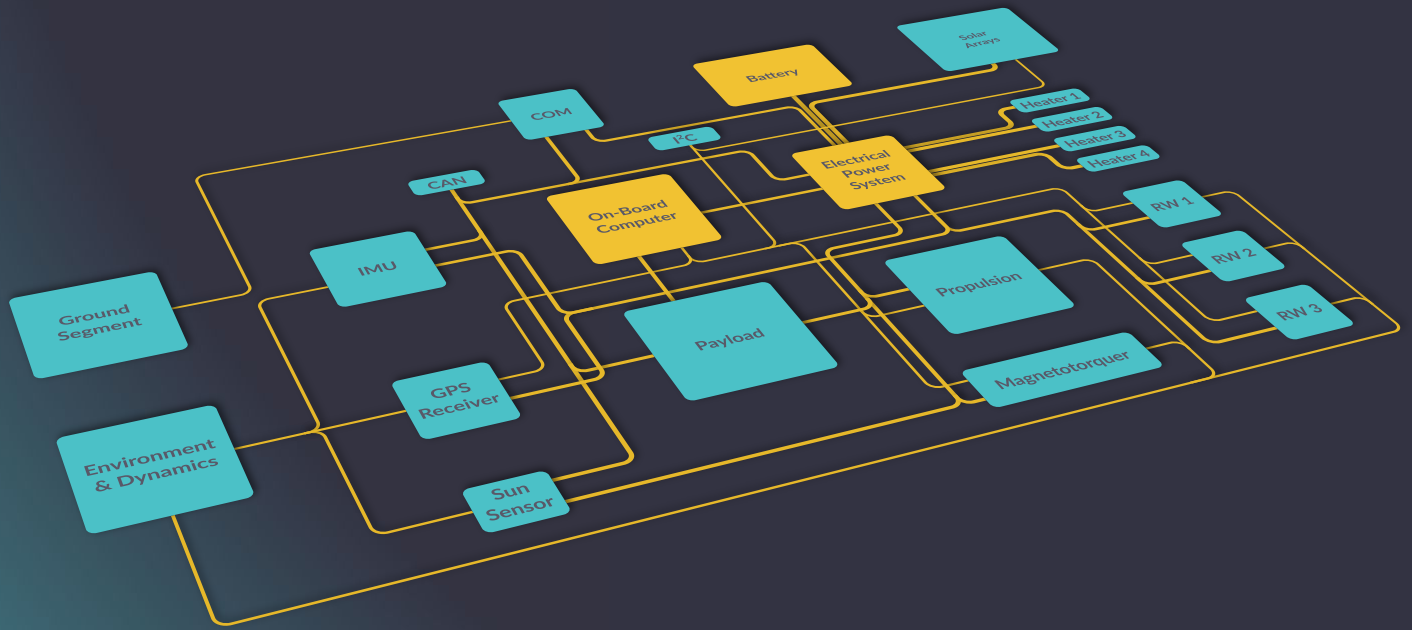
RECAP: HARDWARE-IN-THE-LOOP

asynchronics

Definition: A HIL simulator is a setup where real, physical subsystem instances are immersed within a closed-loop simulation of the remaining subsystems.

In practice: A HIL simulator "fools" the flight hardware by feeding it real-time inputs and outputs as if it were in orbit.

- Simulated
- Real hardware

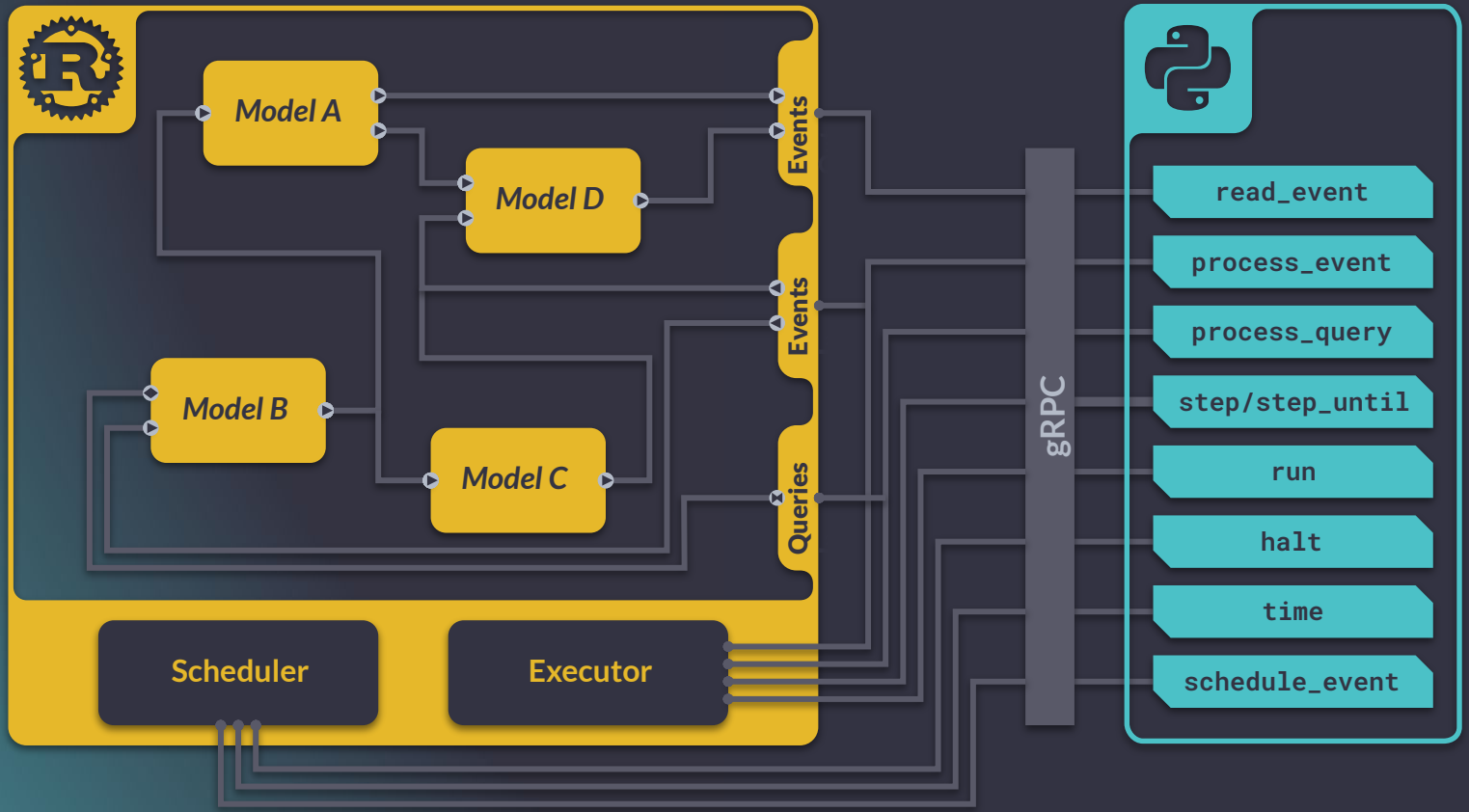


asynchronics

PYTHON API

API: NEXOSIM-PY AT A GLANCE

asynchronics



API: SIMULATION WORKFLOW

asynchronics

1. Define builder function (Rust)

```rust
fn bench(cfg: u16) -> Result<SimInit, Box<dyn Error>> {
    //...
    QuerySource::new()
        .connect(MyModel::my_replier, &model_mbox)
        .bind_endpoint(&mut bench, "replier")?;
    //...
}
```

2. Start server (Rust)

```rust
fn main() {
    server::run(bench, "0.0.0.0:41633".parse().unwrap()).unwrap();
}
```

3. Build/initialize simulation (Python)

```python
from nexosim import Simulation

with Simulation("0.0.0.0:41633") as sim:
        sim.build(123)
        sim.init()
```

4. Run simulation (Python)

```python
print(sim.process_query("replier"))
```

asynchronics

The simulation can be run using one of two server types:

- **Network** server
- **Unix Domain Sockets** local server

Additionally, a `shutdown_signal` Rust future can be hooked into for graceful shutdown.

The server require a `sim_gen` function (closure) that accepts an initialization configuration and builds the simulation with its endpoints.

```
run(sim_gen, addr)?;
```

```
run_with_shutdown(sim_gen, addr, shutdown_signal)?;
```

```
run_local(sim_gen, socket_path)?;
```

```
run_local_with_shutdown(sim_gen, socket_path, shutdown_signal)?;
```

API: SIM_GEN AND BENCH CONFIGURATION

asynchronics

The `sim_gen` function (conventionally named `bench`) is called every time the simulation is (re)started by the remote client to create a new simulation bench and register event and query endpoints.

It takes as argument `cfg`, a simulation configuration with an arbitrary serializable type.

```rust
fn bench(cfg: ModelConfig) -> Result<SimInit, Box<dyn Error>> {
    let model = MyModel::new(cfg);
    let model_mbox = Mailbox::new();

    let mut bench = SimInit::new();

    QuerySource::new()
        .connect(MyModel::my_replier, &model_mbox)
        .bind_endpoint(&mut bench, "replier")?;

    Ok(bench.add_model(model, model_mbox, "model"))
}

fn main() {
    server::run(bench, "0.0.0.0:41633".parse().unwrap()).unwrap();
}
```

```rust
#[derive(Deserialize)]
struct ModelConfig {
    foo: u16,
    bar: String,
}
```

asynchronics

Public endpoints for events (`EventSource`) and queries (`QuerySource`) intended to be **fed** into the simulation must be registered with `bind_endpoint` methods.

Public endpoints for events intended to be **read** from the simulation must implement the `EventSinkReader` trait and be registered with sink-specific functions.

```
EventSource::new()
    .connect(MyModel::my_input, &model_mbox)
    .bind_endpoint(&mut bench, "input")?;
```

```
QuerySource::new()
    .connect(MyModel::my_replier, &model_mbox)
    .bind_endpoint(&mut bench, "replier")?;
```

```
let sink = event_queue_endpoint(&mut bench, SinkState::Enabled, "output")?;
```

```
let sink = event_slot_endpoint(&mut bench, SinkState::Enabled, "output")?;
```

asynchronics

All endpoints must define a **path** used to reference the endpoint on the client side.

Additionally, `EventSinks`, must specify whether they are enabled or disabled by default. The enabled/disabled state of a sink can be changed during the simulation.

```
EventSource::new()
    .connect(MyModel::my_input, &model_mbox)
    .bind_endpoint(&mut bench, "input")?;
```

```
QuerySource::new()
    .connect(MyModel::my_replier, &model_mbox)
    .bind_endpoint(&mut bench, "replier")?;
```

```
let sink = event_queue_endpoint(&mut bench, SinkState::Enabled, "output")?;
```

```
let sink = event_slot_endpoint(&mut bench, SinkState::Enabled, "output")?;
```

API: THE MESSAGE TRAIT

asynchronics

Deriving the `Message` trait on an event/query/reply type is strongly encouraged. It makes it possible for the client to access the **schema** of that type (introspection).

This can be used to **auto-generate** Python types corresponding to the Rust event types.

In the future, it will make it possible to conveniently edit the message in a graphical front-end.

```rust
#[derive(Clone, Serialize, Message)]
pub(crate) struct OutputEvent {
    pub(crate) foo: u16,
    pub(crate) bar: String,
}
```

asynchronics

To start using the simulation from Python with NeXosim-Py, we first need to **connect** to the server.

We do this by using the `Simulation` function, which creates a handle to the remote simulation bench running at the specified address.

```
with Simulation("0.0.0.0:41633") as sim:
  ...
```

asynchronics

Then we **build** the simulation, and **initialize** it with the `build` and `init` methods.

The `build` method calls the `sim_gen` function on the server side, and optionally takes a simulation configuration as argument.

The `init` method takes the starting time of the simulation as an optional argument, which by default is set to `MonotonicTime::EPOCH`.

```
@dataclass
class ModelConfig:
  foo: int
  bar: str

with Simulation("0.0.0.0:41633") as sim:
  sim.build(ModelConfig(123, "string"))
  sim.init()
  print(sim.process_query("replier"))
```

asynchronics

The user configuration passed on the client side must match the configuration defined on the server side!

```python
@dataclass
class ModelConfig:
  foo: int
  bar: str

with Simulation("0.0.0.0:41633") as sim:
  sim.build(ModelConfig(123, "string"))
  sim.init()
  print(sim.process_query("replier"))
```

```rust
#[derive(Deserialize)]
struct ModelConfig {
  foo: u16,
  bar: String,
}
```

API: RUNNING THE SIMULATION

asynchronics

The Python API for advancing the simulation is similar to the Rust API:

- `step`: Advances simulation time to the next scheduled event.
- `step_until`: Iteratively advances the simulation time until the specified deadline.
- `run`: Iteratively advances the simulation time until the simulation ends.
- `halt`: Requests the simulation to stop at the earliest opportunity. Once halted, the simulation can be resumed.

```python
with Simulation("0.0.0.0:41633") as sim:
    sim.build()
    sim.init()
    sim.schedule_event(MonotonicTime(1), "input", 1)
    sim.step()
    print(sim.time())  # 1970-01-01 00:00:01
```

```python
#...
    sim.step_until(MonotonicTime(1))
    sim.step_until(MonotonicTime(2))
    print(sim.time()) # 1970-01-01 00:00:02

    sim.step_until(Duration(2))
    print(sim.time()) # 1970-01-01 00:00:04
```

```python
#...
    sim.schedule_event(MonotonicTime(1), "input", 1)
    sim.schedule_event(MonotonicTime(3), "input", 1)
    sim.run()
    print(sim.time())  # 1970-01-01 00:00:03
```

asynchronics

The simulation is now ready to use! A few other functions exist to manage the lifecycle of the simulation:

- `init_and_run`: Has does the same as `init`, but also initializes the simulation. This is equivalent to first calling `init` and then `run` but avoids network latency. This is useful for latency-sensitive applications such as HiL.
- `close`: Closes the gRPC channel created by the Simulation function. It is only used when not using Python's `with` statement to manually manage resources.
- `terminate`: Terminates the simulation. It destroys the current simulation, `build` and `init` must be used to use the simulation again. It does not close the connection with the server.

API: SENDING & RECEIVING EVENTS & QUERIES

asynchronics

The Python API can send events and make queries in a similar way as in Rust:

- `process_event`: Broadcasts an event from an event source immediately.
- `process_query`: Broadcasts a query from a query source immediately.
- `try_read_events`: Reads all events from an event sink.
- `read_event`: Waits for the next event from an event sink.

```python
with Simulation("0.0.0.0:41633") as sim:
    sim.build()
    sim.init()
    output = sim.process_event("my_input", 5)
```

```python
#...
    output = sim.process_query("my_replier", 5, reply_type=str)
    print(output)
```

API: SENDING & RECEIVING EVENTS & QUERIES

asynchronics

```
with Simulation("0.0.0.0:41633") as sim:
    sim.build()
    sim.init()
    output = sim.process_event("my_input", 5)
```

```
#...
    output = sim.process_query("my_replier", 5, reply_type=str)
    print(output)
```

The Python API also makes it possible to **enable and disable sinks**.

```
sim.enable_sink("sim")
```

```
sim.disable_sink("sink")
```

asynchronics

try_read_events

```python
@dataclass
class OutputEvent:
    foo: int
    bar: str

with Simulation(address='localhost:41633') as sim:
    sim.build()
    sim.init()

    sim.process_event("input", 1)
    outputs = sim.try_read_events("output", OutputEvent)
    print(f"Events mapped to the OutputEvent class: {outputs}")

    sim.process_event("input", 1)
    outputs = sim.try_read_events("output")
    print(f"Events without specifying the reply type: {outputs}")

# Prints out:
# Events mapped to the OutputEvent class: [OutputEvent(foo=1,
bar='string')]
# Events without specifying the reply type: [{'foo': 1, 'bar': 'string'}]
```

API: SENDING & RECEIVING EVENTS & QUERIES

asynchronics

read_event

```python
from dataclasses import dataclass
from nexosim import Simulation
from nexosim.time import Duration

@dataclass
class OutputEvent:
    foo: int
    bar: str

with Simulation(address='localhost:41633') as sim:
    sim.build()
    sim.init()

    sim.process_event("input", 1)
    output = sim.read_event("output", Duration(1), OutputEvent)
    print(output)

# Prints out:
# OutputEvent(foo=1, bar='string')
```

API: SCHEDULING EVENTS

asynchronics

Events can be scheduled using the `schedule_event` method.

By setting its parameters, we can configure whether we want to schedule periodic or non-periodic events, and whether we want to acquire a key to the event to be able to cancel it.

For event cancellation, we can use the `cancel_event` method.

```
with Simulation("0.0.0.0:41633") as sim:
    sim.build()
    sim.init()
    event_key = sim.schedule_event(Duration(10), "my_event", with_key=True)
    sim.cancel_event(event_key)
```

NEXOSIM CLIENT API: SAVE & RESTORE

asynchronics

Just as in Rust, in Python we can also save and restore our simulation. For this purpose, the `save` and `restore` methods exist.

```python
with Simulation(address='localhost:41633') as sim:
    sim.build()
    sim.init()

    sim.process_event("input")

    state = sim.save()
    sim.terminate()
    sim.build()
    sim.restore(state)

    sim.process_event("input")
    outputs = sim.try_read_events("output")
    print(outputs)

# Prints:
# [2]
```

NEXOSIM CLIENT API: INJECTING EVENTS

asynchronics

We can also inject events into our simulation using the `inject_event` method.

Injecting events means scheduling them to be processed at the earliest possible time. This will be the time of the next tick (if the simulation was set up with a ticker) or the time of the next scheduled event, whichever occurs first.

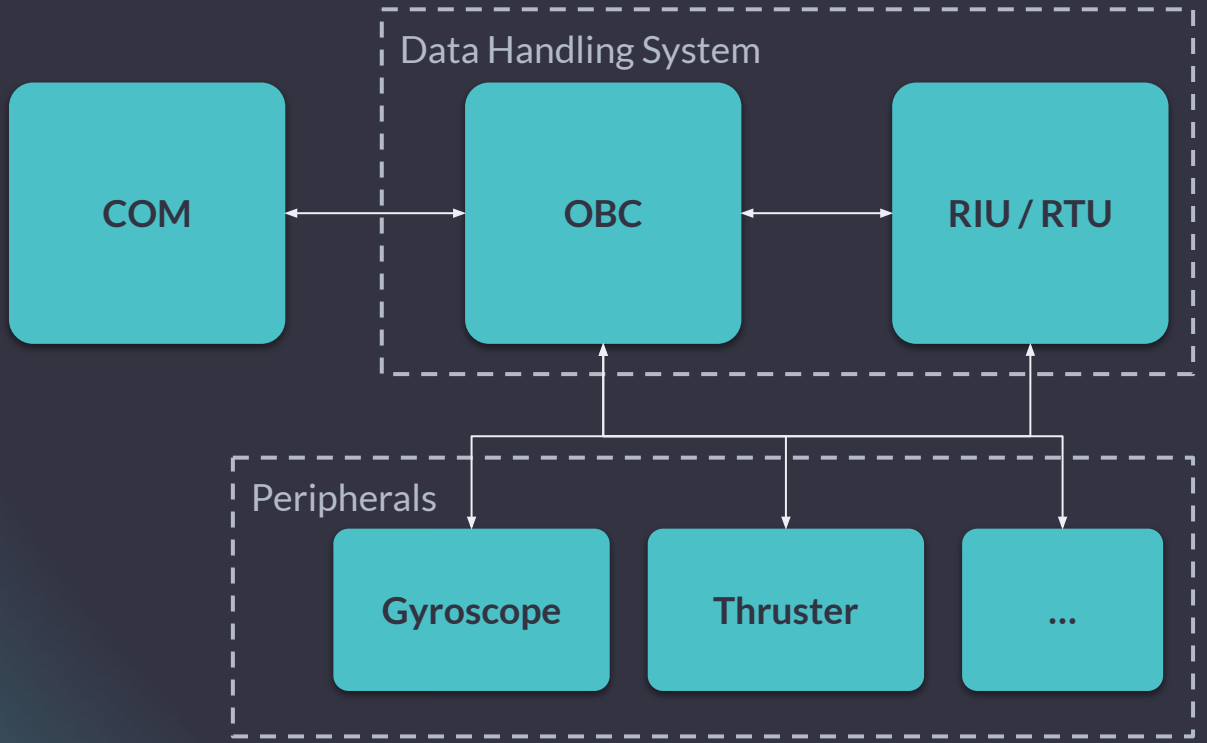
```
with Simulation("0.0.0.0:41633") as sim:
    sim.build()
    sim.init()
    sim.inject_event("my_input", 5)
```

asynchronics

REAL SETUP EXAMPLE

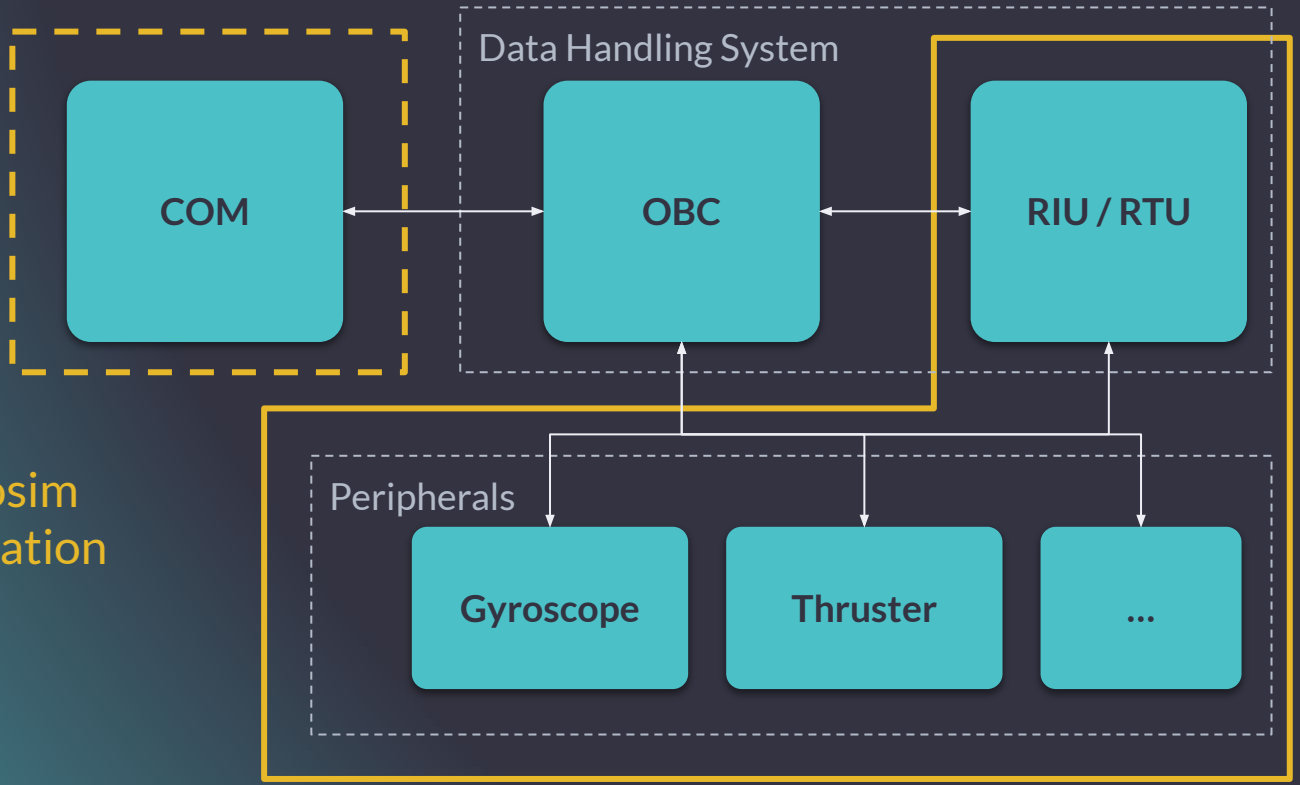
EXAMPLE: ARCHITECTURE OVERVIEW

asynchronics



EXAMPLE: MAPPING THE SYSTEM TO A SIMULATION

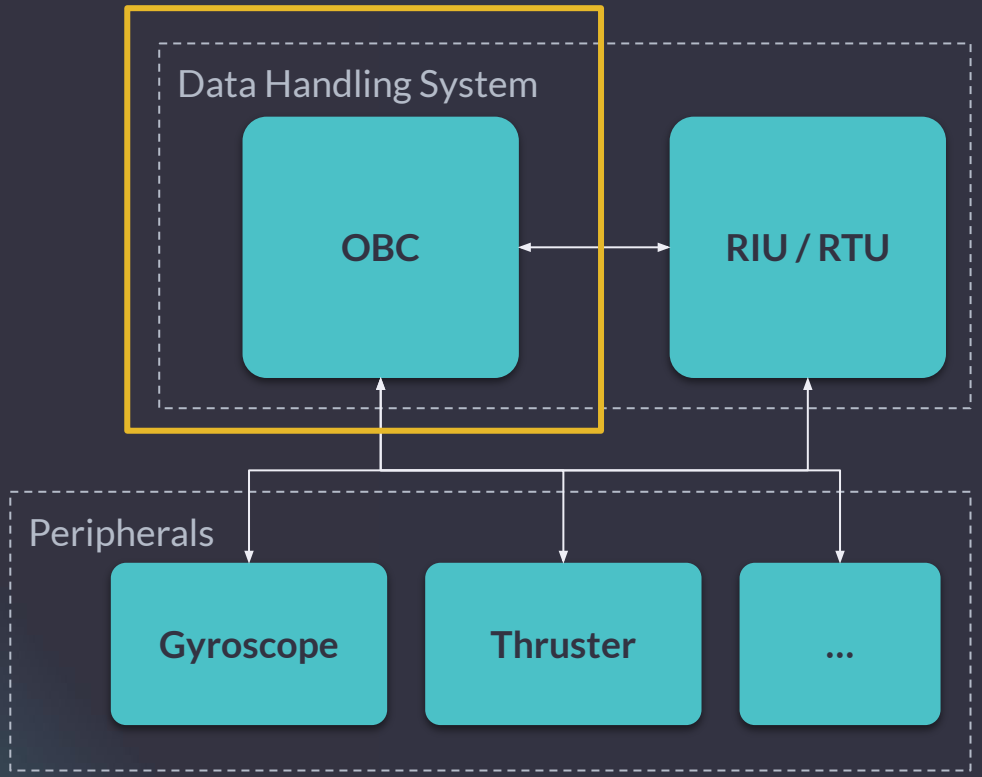
asynchronics



NeXosim simulation

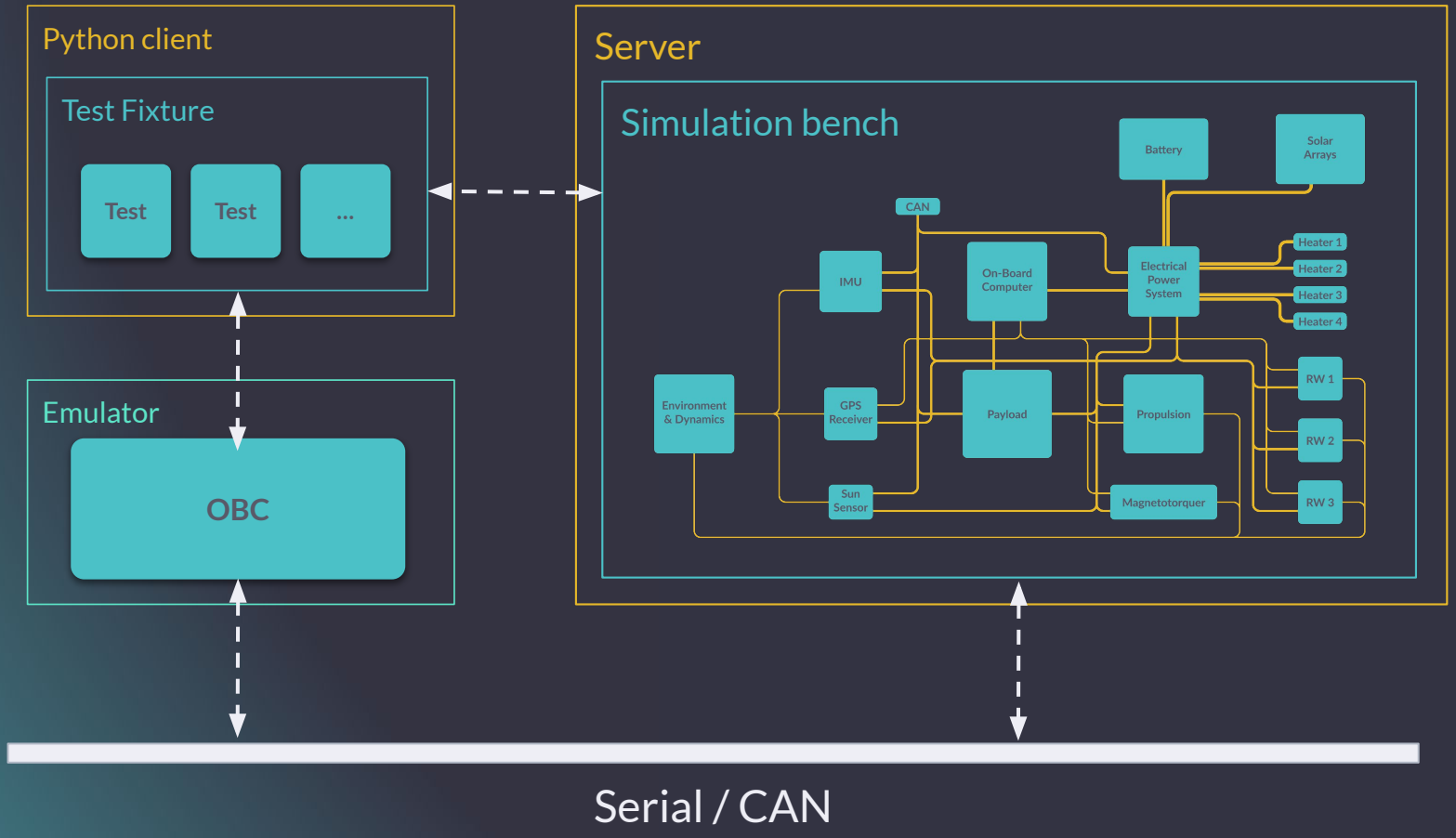
asynchronics

HITL or emulation



EXAMPLE: TYING IT TOGETHER

asynchronics



asynchronics

BEST PRACTICES

asynchronics

Where should the configuration be stored?

Some possibilities:

- database
- server-local files
- **client-local files**

asynchronics

For client-local file configuration, the easiest is to **send the "raw" configuration** payload, e.g. as text or binary from the client, that is:

raw config → Rust

rather than:

raw config → Python → Rust

```
with open("config/gyro_config.toml", "r") as file:
    gyro_cfg = file.read()
with open("config/rw_config.toml", "r") as file:
    rw_cfg = file.read()

cfg = (gyro_cfg, rw_cfg)

with Simulation("localhost:41633") as sim:
    sim.build(cfg)

    # ...
```

asynchronics

For client-local file configuration, the easiest is to **send the "raw" configuration** payload, e.g. as text or binary from the client, that is:

raw config → Rust

rather than:

raw config → Python → Rust

```rust
type BenchConfig = (String, String);

fn bench(cfg: BenchConfig) -> Result<SimInit, Box<dyn Error>> {
    let (gyro_cfg, rw_cfg) = cfg;

    let mut gyro_cfg_loader = ConfigLoader::<GyroConfig>::new();
    gyro_cfg_loader.code(gyro_cfg, Format::Toml)?;
    let mut gyro = Gyro::new(gyro_cfg_loader.load()?.config);

    let mut rw_cfg_loader = ConfigLoader::<ReactionWheelConfig>::new();
    rw_cfg_loader.code(rw_cfg, Format::Toml)?;
    let mut rw = ReactionWheel::new(rw_cfg_loader.load()?.config);

    // ...
}
```

asynchronics

Predefined bench topologies can be dynamically selected from the remote client by sending the appropriate configuration in the `build()` method.

```rust
#[derive(Deserialize)]
enum Topology {
    /// Topology with two different loads.
    AB(f64, f64),
    /// Topology with multiple identical loads
    Many(u8, f64),
}
// ...

fn bench(topology: Topology) -> Result<SimInit, Box<dyn Error>> {
    let mut ps = PowerSupply::default();
    let ps_mbox = Mailbox::new();

    let mut bench = SimInit::new();


    match topology {

        Topology::AB(resistance_a, resistance_b) => {
            // ...
        }
        Topology::Many(n, resistance) => {
            // ...
        }
    }

    Ok(bench.add_model(ps, ps_mbox, "POWER_SUPPLY"))
```

asynchronics

Predefined bench topologies can be dynamically selected from the remote client by sending the appropriate configuration in the `build()` method.

```
Topology::AB(resistance_a, resistance_b) => {
    let mut load_a = Load::new(resistance_a);
    let load_a_mbox = Mailbox::new();

    let mut load_b = Load::new(resistance_b);
    let load_b_mbox = Mailbox::new();

    ps.power_out.connect(Load::power_in, &load_a_mbox);
    ps.power_out.connect(Load::power_in, &load_b_mbox);

    let sink = event_queue_endpoint(&mut bench, SinkState::Enabled,
"power_A")?;
    load_a.power.connect_sink(sink);

    let sink = event_queue_endpoint(&mut bench, SinkState::Enabled,
"power_B")?;
    load_b.power.connect_sink(sink);

    bench = bench.add_model(load_a, load_a_mbox, "Load_A").add_model(
        load_b,
        load_b_mbox,
        "Load_B",
    );
}
```

asynchronics

Predefined bench topologies can be dynamically selected from the remote client by sending the appropriate configuration in the `build()` method.

```
Topology::Many(n, resistance) => {
    let sink = event_queue_endpoint(
        &mut bench, SinkState::Enabled, "load_power")?;

    for i in (0..n).into_iter() {
        let mut load = Load::new(resistance);
        let load_mbox = Mailbox::new();

        ps.power_out.connect(Load::power_in, &load_mbox);
        load.power.connect_sink(sink.clone());

        bench = bench.add_model(load, load_mbox, &(format!("Load_{}", i +
1)));
    }
}
```

asynchronics

Predefined bench topologies can be dynamically selected from the remote client by sending the appropriate configuration in the `build()` method.

```
from nexosim import Simulation
from nexosim.types import enumclass, tuple_type

@enumclass
class Topology:
   class AB(tuple_type(float, float)): ...
   class Many(tuple_type(int, float)): ...

   type = AB | Many


with Simulation("localhost:41633") as sim:

   cfg = Topology.AB(100.0, 500.0)

   sim.build(cfg)
   sim.init()
```

asynchronics

To inject faults into the simulation via a remote client, such as disabling connections between models, we can define a model with input ports that toggle switches controlling specific connections.

Since the switch is an `AtomicBool` which cannot be serialized, it has to be stored in the model's `Env`.

```rust
struct FaultManagerEnv {
    tmtc_switch: Arc<AtomicBool>,
}

#[derive(Serialize, Deserialize)]
struct FaultManager;

#[Model(type Env=FaultManagerEnv)]
impl FaultManager {
    fn toggle_tmtc_fault(
        &mut self,
        state: bool,
        _: &Context<Self>,
        env: &mut <Self as Model>::Env,
    ) {
        env.tmtc_switch.store(state, Ordering::Relaxed);
    }
}
```

asynchronics

We need to define a simple model prototype, to allow us to provide the `AtomicBool` reference that will be kept in the model environment.

```
struct ProtoFaultManager {
    tmtc_switch: Arc<AtomicBool>,
}

impl ProtoModel for ProtoFaultManager {
    type Model = FaultManager;

    fn build(
        self,
        _: &mut nexosim::model::BuildContext<Self>,
    ) -> (Self::Model, <Self::Model as Model>::Env) {
        (
            FaultManager,
            FaultManagerEnv {
                tmtc_switch: self.tmtc_switch,
            },
        )
    }
}
```

asynchronics

To disable the connection based on the bool state we move a clone of the switch into the filtering closure and check its value. If true all incoming events are ignored.

The switch is then moved to the `ProtoFaultManager` and an `EventSource` is connected to the `toggle_tmtc_fault()` input.

```
let switch = Arc::new(AtomicBool::new(false));

let switch_clone = switch.clone();
obc.tc.filter_map_connect(
    move |e| {
        if !switch_clone.load(Ordering::Relaxed) {
            Some(e.clone())
        } else {
            None
        }
    },
    Device::tc,
    &device_mbox,
);

// ...

let fault_manager = ProtoFaultManager {
    tmtc_switch: switch,
};

EventSource::new()
    .connect(FaultManager::toggle_tmtc_fault, &fm_mbox)
    .bind_endpoint(&mut bench, "toggle_tmtc_fault")?;
```

asynchronics

Timestamping events could be useful for tracing numeric parameters of models.

Events can be timestamped using a mapped connection by reading the time using `ClockReader::time()`.

The `ClockReader` can be retrieved with `SimInit::clock_reader()`.

```
let yaw_sink = event_queue_endpoint(&mut bench, SinkState::Enabled, "yaw")?;
let clock = bench.clock_reader();
gyro.yaw.map_connect_sink(move |e| (clock.time(), *e), yaw_sink);
```

```
with Simulation("...") as sim:
    # ...
    yaw = sim.try_read_events("yaw", tuple[MonotonicTime, float])
```

asynchronics

Communication with a database from a model can be accomplished using a client in a separate thread.

The thread handle is stored in the model's Environment.

```rust
#[derive(Serialize, Deserialize)]
pub struct DbAdapter;

#[Model(type Env=DbEnv)]
impl DbAdapter {
    /// This method should not be used on untrusted sources
    /// as it's prone to SQL injection attacks.
    pub fn execute_query(&mut self, stmt: String,
                         _: &Context<Self>, env: &mut DbEnv) {
        if let Some(tx) = &env.tx {
            let _ = tx.try_send(stmt);
        }
    }
}

pub struct DbEnv {
    tx: Option<SyncSender<String>>,
    handle: Option<std::thread::JoinHandle<()>>,
}
impl Drop for DbEnv {
    fn drop(&mut self) {
        // Drop sender to disconnect the channel.
        self.tx = None;
        if let Some(handle) = self.handle.take() {
            handle.join().unwrap();
        }
    }
}
```

asynchronics

Communication with a database from a model can be accomplished using a client in a separate thread.

The thread handle is stored in the model's Environment.

```rust
pub struct ProtoDb {
    connection_str: String,
}
impl ProtoDb {
    pub fn new(connection_str: String) -> Self {
        Self { connection_str }
    }
}
impl ProtoModel for ProtoDb {
    type Model = DbAdapter;
    fn build(
        self,
        _: &mut nexosim::model::BuildContext<Self>,
    ) -> (Self::Model, <Self::Model as nexosim::model::Model>::Env) {
        let (tx, rx) = sync_channel(QUEUE_SIZE);
        let handle = std::thread::spawn(move || {
            db_thread(rx, self.connection_str);
        });
        (
            DbAdapter,
            DbEnv {
                tx: Some(tx),
                handle: Some(handle),
            },
        )
    }
}
```

asynchronics

Upon receiving data (an SQL statement) through a channel, the thread first checks if the client exists and, if necessary, connects the client to the database.

It then attempts to send the received data to the database.

The client is dropped if it was disconnected from the database.

```
fn db_thread(rx: Receiver<String>, connection_str: String) {
    let mut client = None;
    let mut last_reconnect =
        std::time::Instant::now() - RECONNECT_INTERVAL;
    while let Ok(stmt) = rx.recv() {
        if client.is_none() {
            if last_reconnect.elapsed() < RECONNECT_INTERVAL {
                continue;
            }
            match postgres::Client::connect(
                &connection_str, postgres::NoTls) {
                Err(e) => {
                    last_reconnect = std::time::Instant::now();
                    continue;
                }
                Ok(c) => {
                    client = Some(c);
                }
            }
        }
        if let Err(e) = client.as_mut().unwrap().execute(&stmt, &[]) {
            // Check for a closed connection
            if e.is_closed() {
                client = None;
            }
        };
    }
}
```

asynchronics

To write to the database, we can simply use a mapped connection where we create an SQL statement using the broadcast events.

```
model.out.map_connect(
    |e| format!("INSERT INTO table (val) VALUES ({})", e),
    DbAdapter::execute_query,
    &db_mbox,
);
```

asynchronics

Questions